Fondamenti di Grafica Tridimensionale

Laurea Magistrale in Ingegneria Informatica e delle Tecnologie dell'Informazione

Docente Dr. Ugo Erra

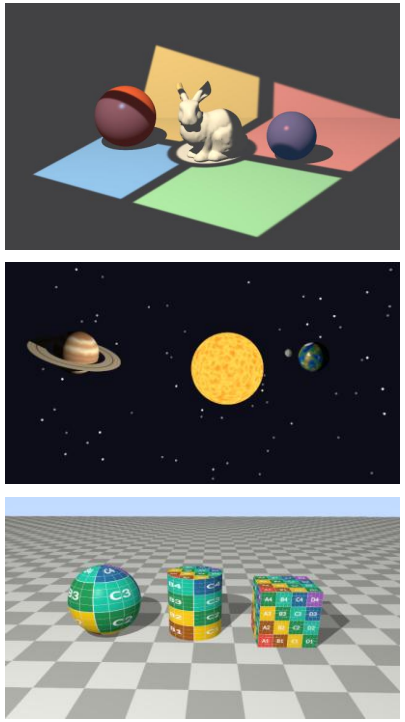
II° Homeworks

Homeworks

- Luci e ombre
- Trasformazioni e movimento
- Texture e superfici
- Il progetto finale
- Modalità di consegna

Come funziona

- Si parte dal progetto della **lezione 10** (`Texture Mesh`, con TinyBVH): Phong, ombre, istanze, mesh e texture ci sono già
- Ogni scheda dice **che cosa fare**, la matematica essenziale e **che cosa consegnare**
- L'indicatore in alto a destra dà la difficoltà: **facile** (si estende il codice che avete), **medio** (una formula nuova e i suoi casi limite), **difficile** (più pezzi nuovi da far funzionare insieme)
- Alcuni homework ne usano altri: il proiettore (2) parte dal faretto (1); il motion blur (5) e il sistema solare (6) usano le texture delle primitive (8); il cortometraggio usa l'1, il 3 e l'8
- Finché la parte nuova non è corretta lavorate a bassa risoluzione e con un campione per pixel



tre risultati possibili

Le immagini di queste slide

- sono prodotte da un ray tracer di riferimento che implementa gli homework: mostrano il risultato atteso, non un traguardo estetico
- i tempi e i numeri citati sono misurati su quella implementazione o calcolati; i vostri saranno diversi, ma l'andamento deve tornare

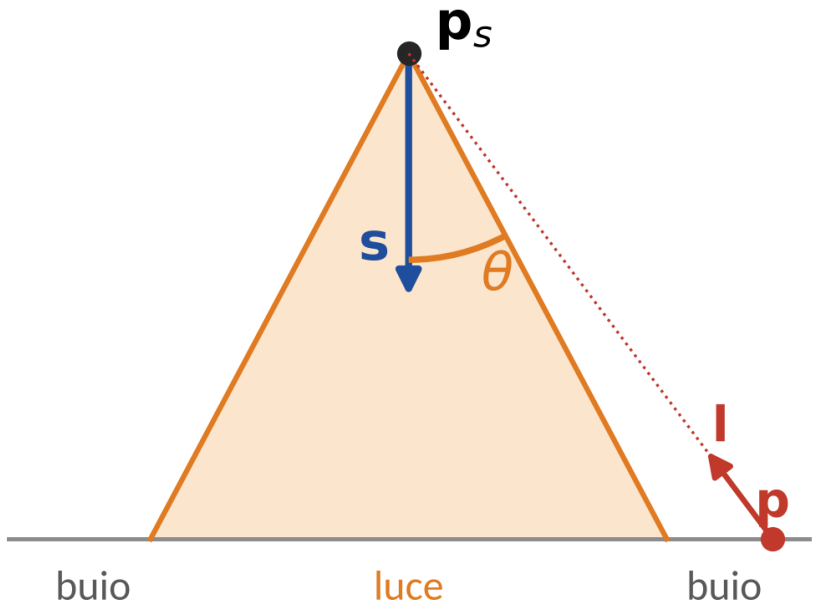
Homeworks

- **Luci e ombre**
- Trasformazioni e movimento
- Texture e superfici
- Il progetto finale
- Modalità di consegna

1 · Un faretto semplice

●○○ facile

- Un **faretto** (spot) è una luce puntiforme che illumina solo dentro un cono: posizione $\mathbf{p}_s$, direzione **s** (unitaria, verso cui punta) e un **semiangolo** ϑ
- ϑ è misurato dall'asse al bordo: con $\vartheta = 30°$ il cono è largo $2 \cdot \vartheta = 60°$ (lezione 6)
- Per un punto **p** si calcola **l**, il vettore unitario dal punto verso la luce, come per la luce puntiforme. Il punto è nel cono se l'angolo fra $-\mathbf{l}$ e **s** è minore di ϑ
- Niente arcocoseni: basta confrontare $-\mathbf{l} \cdot \mathbf{s}$ con $\cos \vartheta$ (il coseno decresce, quindi angolo più piccolo vuol dire coseno più grande)
- Fuori dal cono resta solo l'ambientale; dentro, tutto come la luce puntiforme della lezione 6, ombre comprese

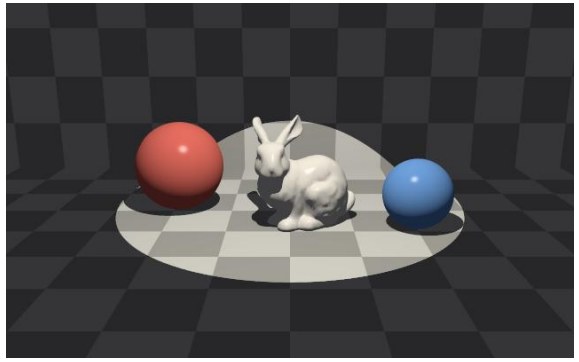


il punto p è fuori dal cono: lì il faretto non illumina

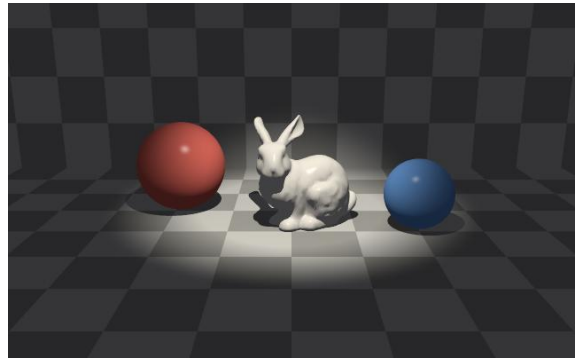
$$f_{spot} = \begin{cases} 1 & \text{se } -\mathbf{l} \cdot \mathbf{s} \geq \cos\theta \ \text{ (dentro il cono)} \\ 0 & \text{altrimenti} \end{cases} \qquad I = k_a L_a + f_{spot}\, V\, [k_d L_d \max(\mathbf{n} \cdot \mathbf{l}, 0) + k_s L_s \max(\mathbf{r} \cdot \mathbf{v}, 0)^{\alpha}]$$

1 · Che cosa costruire

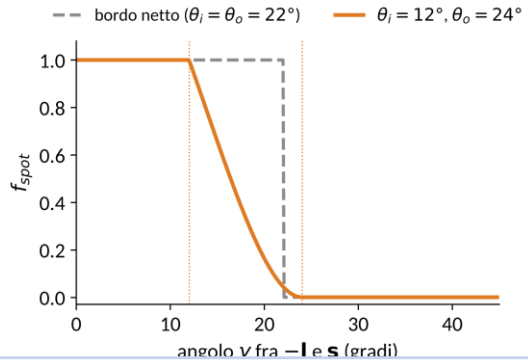
- Scrivere `spot_light` come sottoclasse di `light`, con $\mathbf{p}_s$, $\mathbf{s}$ e ϑ
- Conviene dare a `light` un metodo virtuale che, dato $\mathbf{p}$, restituisca $\mathbf{l}$, la distanza e l'intensità: lo shading non deve sapere che luce ha davanti (servirà anche negli homework 2 e 3)
- Scena di prova: pavimento, il coniglio, due sfere e il solo faretto, puntato dall'alto
- Per andare oltre: il **bordo sfumato** della lezione 6 con due semiangoli $\vartheta_i \le \vartheta_o$ e f_{spot} che scende da 1 a 0 fra i due coni (con $\vartheta_i = \vartheta_o$ si torna al bordo netto)



bordo netto: θ = 22°



sfumato: θ_i = 12°, θ_o = 24°



Che cosa consegnare

- `spot_light` integrato nello shading, con il test sul cono
- il render della scena con il bordo netto e, se fatto, quello con il bordo sfumato

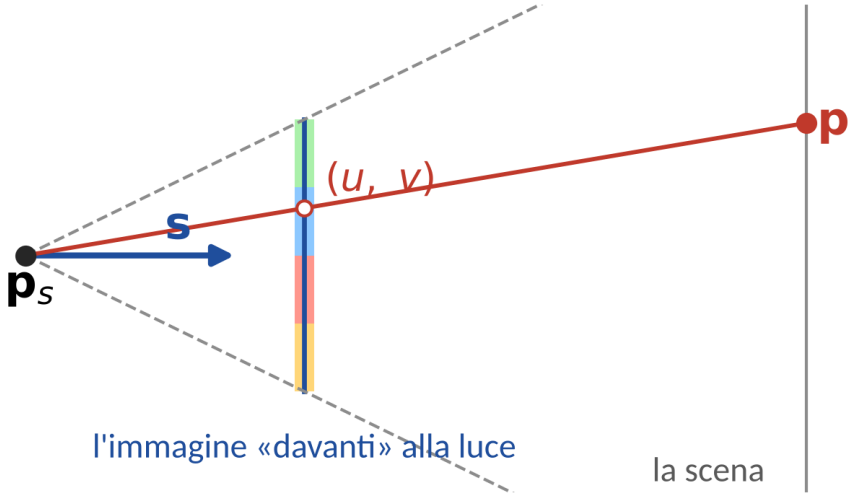
2 · Proiettare un'immagine con un faretto

●●○ medio

- Un solo faretto, che invece di una luce uniforme proietta un'immagine, come un proiettore di diapositive: il colore della luce cambia con la direzione
- Attorno a **s** si costruisce una base **a**, **b** come **u**, **v** della camera con `vup` (lezione 5); se **s** è parallelo a `vup` (faretto verticale) serve un altro `vup`
- Il punto **p** si proietta sul piano davanti alla luce: è la camera della lezione 5 usata al contrario, dal punto al pixel
- Buio se $\mathbf{q} \cdot \mathbf{s} \le 0$ (il punto è dietro la luce) o se (u, v) cade fuori da $[0, 1]^2$; altrimenti L_d è moltiplicata per `texture::value(u, v, p)` (lezione 10)
- Il cono del faretto ora è il quadrato dell'immagine: ϑ fissa quanto è larga la proiezione

$$\mathbf{a} = \frac{\mathbf{s} \times \mathbf{vup}}{\|\mathbf{s} \times \mathbf{vup}\|}, \quad \mathbf{b} = \mathbf{a} \times \mathbf{s}, \quad \mathbf{q} = \mathbf{p} - \mathbf{p}_s$$

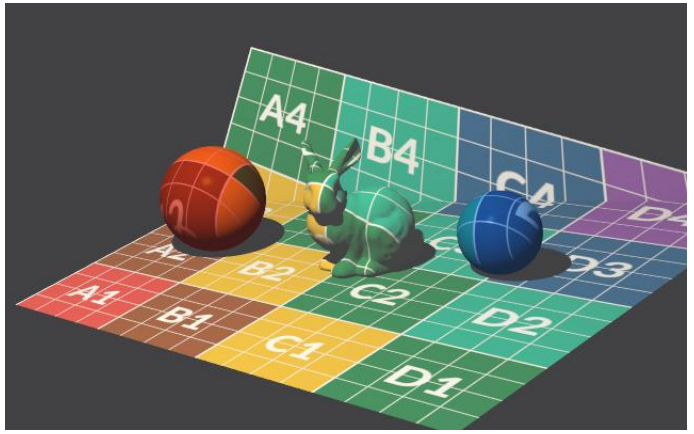
$$u = \frac{1}{2} + \frac{1}{2}\frac{\mathbf{q} \cdot \mathbf{a}}{(\mathbf{q} \cdot \mathbf{s}) \tan \theta_o}, \quad v = \frac{1}{2} + \frac{1}{2}\frac{\mathbf{q} \cdot \mathbf{b}}{(\mathbf{q} \cdot \mathbf{s}) \tan \theta_o}$$



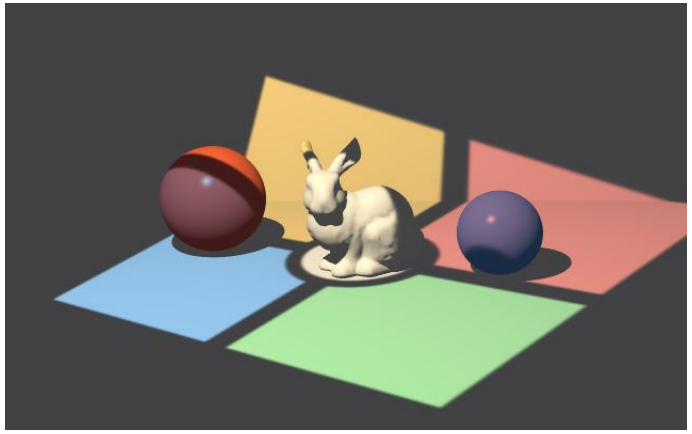
il punto p prende il colore del pixel (u, v)

2 · Che cosa costruire

- Aggiungere a `spot_light` una texture facoltativa: con `nullptr` resta il faretto dell'homework 1
- Prima un'immagine con le lettere: si vede subito se l'immagine esce ribaltata o specchiata
- Poi un'immagine a scelta: una vetrata, una diapositiva, la proiezione di un film su una parete
- Le ombre non costano niente di più: lo shadow ray verso $\mathbf{p}_s$ è lo stesso del faretto



la griglia con le lettere


una vetrata

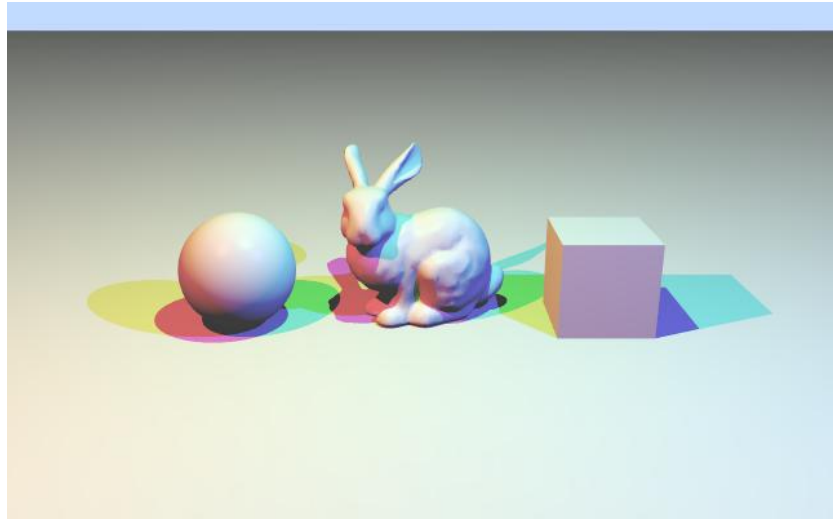
Che cosa consegnare

- il faretto con la texture, in una scena con un solo faretto e nessun'altra luce oltre l'ambientale
- due render: con la griglia di lettere e con un'immagine a scelta

3 · Più sorgenti di luce

●●○ medio

- Oggi `ray_color` riceve una sola `point_light`: deve ricevere una **lista** di luci
- Per ogni luce un suo shadow ray e un suo V_i (0 in ombra, 1 altrimenti); i contributi si sommano
- L'ambientale si conta **una volta sola**: va spostato dalla luce alla scena (una sola L_a, lezione 6)
- Nella lista convivono luci diverse: puntiforme, **direzionale** (**l** costante, shadow ray senza limite di distanza) e il faretto dell'homework 1; per le prime due $f_{spot} = 1$
- Con più luci il colore supera 1 e `setColor` taglia ogni canale: la tinta vira al bianco. Conviene attenuare le luci (per esempio dividere per il loro numero)



tre luci puntiformi rossa, verde e blu: ogni ombra manca di un colore

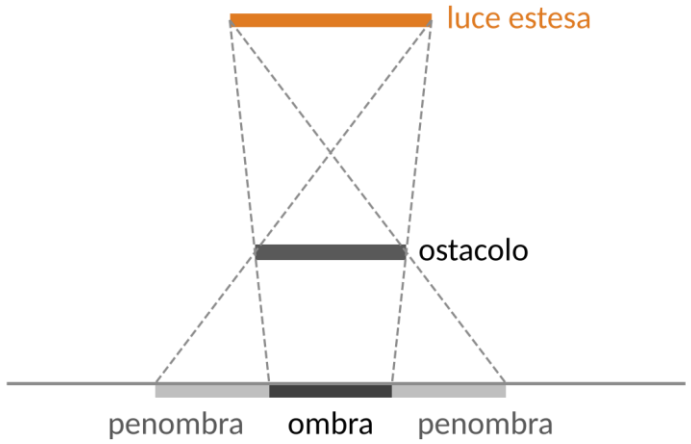
$$I = k_a L_a + \sum_{i=1}^{n} V_i \, f_{spot,i} \, [k_d L_{d,i} \max(\mathbf{n} \cdot \mathbf{l}_i, 0) + k_s L_{s,i} \max(\mathbf{r}_i \cdot \mathbf{v}, 0)^{\alpha}]$$

Che cosa consegnare

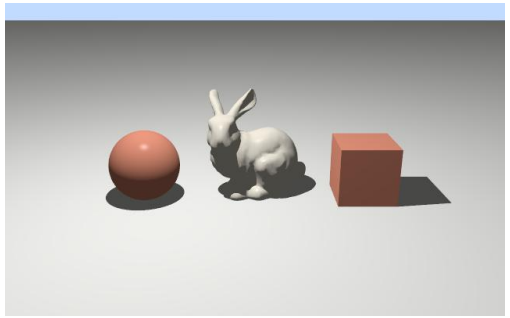
- la lista di luci con almeno tre tipi (puntiforme, direzionale, spot) e la luce estesa della slide seguente
- il render delle tre luci colorate e i quattro render delle ombre morbide con il tempo di ciascuno

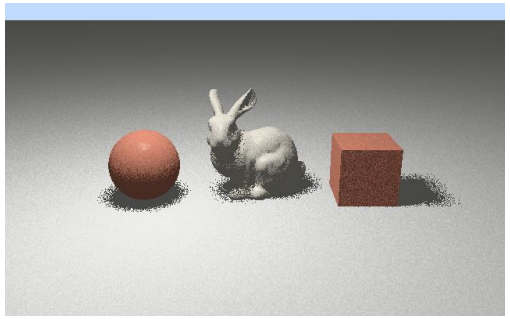
3 · Una luce estesa: le ombre morbide

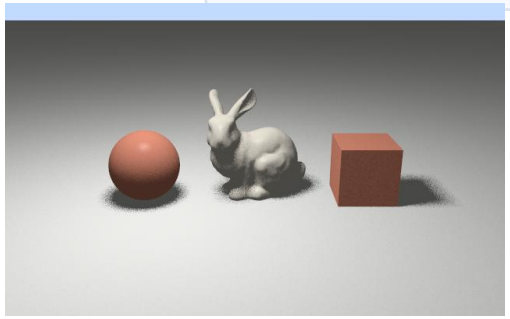
- Una luce vera ha una superficie: da un punto in **penombra** se ne vede solo una parte
- Luce rettangolare con vertice **Q** e lati **u**, **v**: lo stesso quadrilatero dell'homework 1 della I° serie
- Per ogni punto si scelgono *N* punti a caso sulla luce e si lancia uno shadow ray verso ciascuno: la frazione che passa è la visibilità *V*
- Costo: *N* shadow ray per campione. Si risparmia usando i campioni per pixel che ci sono già: a ogni campione un punto diverso della luce
- Se il quadrilatero della luce è anche un oggetto della scena va escluso dagli shadow ray, altrimenti ogni punto risulta in ombra

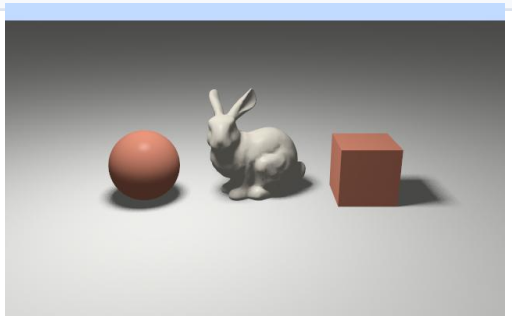


$$\mathbf{y}_k = \mathbf{Q} + \xi_1 \mathbf{u} + \xi_2 \mathbf{v}, \quad \xi_1, \xi_2 \in [0, 1) \qquad V \approx \frac{1}{N} \sum_{k=1}^{N} V(\mathbf{p}, \mathbf{y}_k)$$


luce puntiforme


estesa, N = 1

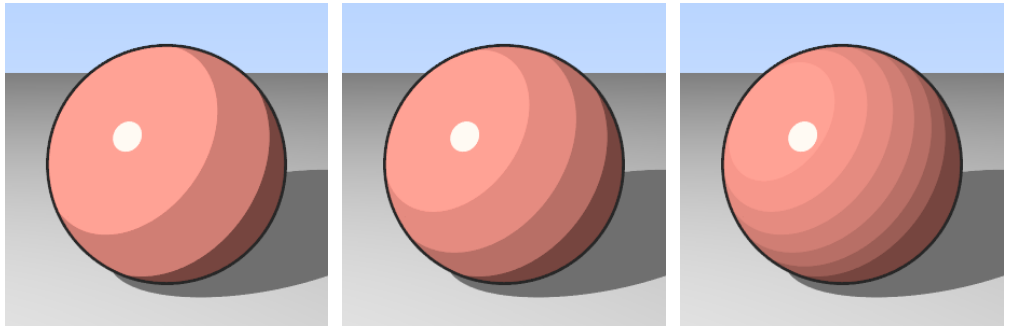

N = 16


N = 256

4 · Un cartone animato: K e d

●○○ facile

- Nel **toon shading** la luce diffusa non sfuma: la superficie è divisa in poche zone di colore piatto, come in un cartone animato
- *K* è il **numero di livelli** di luce, un intero scelto da voi (di solito 2, 3 o 4)
- *d* prende il posto di max(**n** · **l**, 0) nel termine diffuso di Phong: si moltiplica **n** · **l** per *K*, si arrotonda per eccesso e si divide per *K*
- Così *d* può valere solo 1/*K*, 2/*K*, …, 1, e 0 dove **n** · **l** ≤ 0 (la parte non illuminata, con il solo ambientale)
- Con *K* = 1 il lato illuminato è tutto uguale; al crescere di *K* le zone si stringono e si torna verso Phong



K = 2 K = 3 K = 6

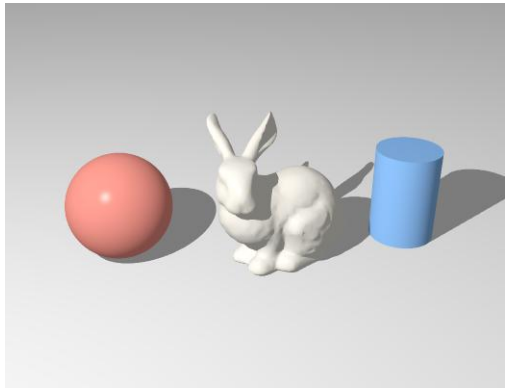
$\mathbf{n}\cdot\mathbf{l}$	≤ 0	0.2	0.5	0.7	0.9
$K\,(\mathbf{n}\cdot\mathbf{l})$	–	0.6	1.5	2.1	2.7
$\lceil\cdot\rceil$	0	1	2	3	3
d	0	1/3	2/3	1	1

$(K = 3)$

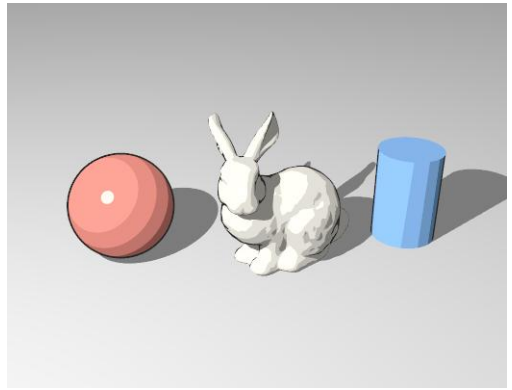
$$d = \frac{\lceil K \cdot \max(\mathbf{n}\cdot\mathbf{l},\ 0)\rceil}{K} \qquad I = k_a L_a + V\, k_d L_d\, d \ + \ (\text{macchia speculare})$$

4 · Contorno, riflesso e che cosa costruire

- La speculare diventa una macchia: piena se **r** · **v** supera una soglia (per esempio 0.97), altrimenti nulla
- Il **contorno**: dove la superficie è vista di taglio $|\mathbf{n} \cdot \mathbf{v}|$ è piccolo; sotto una soglia ε il pixel diventa nero
- Funziona sulle superfici curve. Sul pavimento visto radente annerirebbe l'orizzonte: lì il toon va spento con un flag nel materiale
- Ombre portate e luci restano quelle di prima: cambia solo il modo di calcolare il colore



Phong



toon: K = 3, ε = 0.26

$$d = \frac{\lceil K \max(\mathbf{n} \cdot \mathbf{l}, 0) \rceil}{K} \quad (K \text{ livelli}) \qquad \text{contorno se } |\mathbf{n} \cdot \mathbf{v}| < \varepsilon$$

Che cosa consegnare

- un flag (o una sottoclasse) di `material` che attiva il toon, con *K* e ε nel costruttore
- lo stesso render in Phong e in toon con *K* = 2, 3 e 5, e una riga: dove sbaglia il contorno calcolato con $|\mathbf{n} \cdot \mathbf{v}|$?

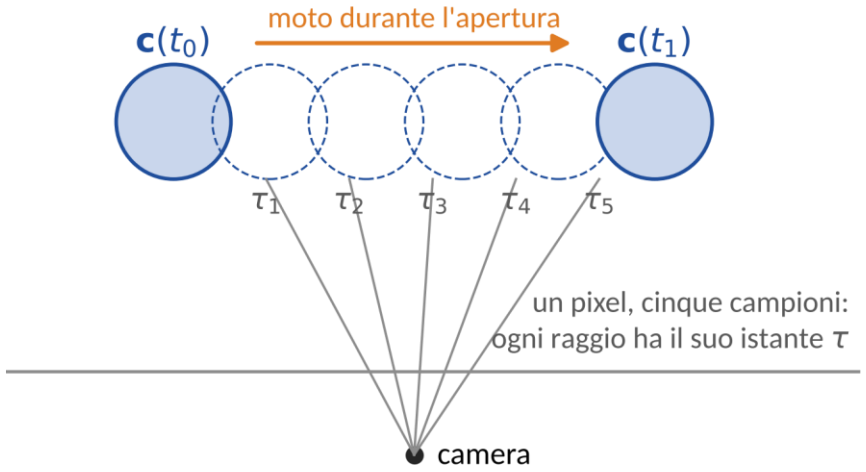
Homeworks

- Luci e ombre
- **Trasformazioni e movimento**
- Texture e superfici
- Il progetto finale
- Modalità di consegna

5 · Motion blur: il tempo di posa

●●○ medio

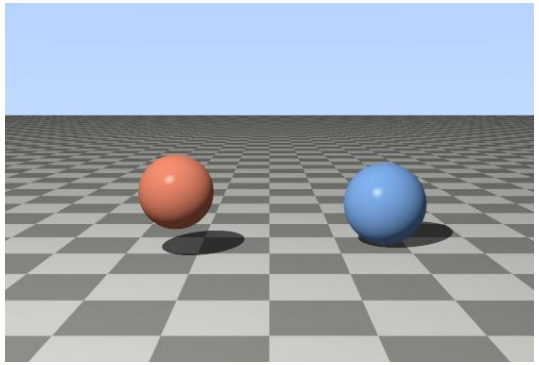
- Una fotocamera tiene l'otturatore aperto per un intervallo $[t_0, t_1]$: un oggetto che si muove lascia una scia
- Ogni raggio porta con sé un istante τ scelto a caso nell'intervallo: un campo in più nella classe `ray`
- L'oggetto risponde con la posizione che ha in quell'istante: per una sfera basta interpolare il centro
- Con le istanze (lezione 7) servono due trasformazioni, a t_0 e a t_1: per una traslazione si interpola il vettore, per una rotazione l'**angolo**, mai gli elementi della matrice
- Lo shadow ray usa lo stesso τ del raggio primario, e così il raggio trasformato in `instance::hit`: se si perde per strada, l'ombra non segue l'oggetto
- Servono molti campioni per pixel: la media sui τ approssima l'integrale nel tempo



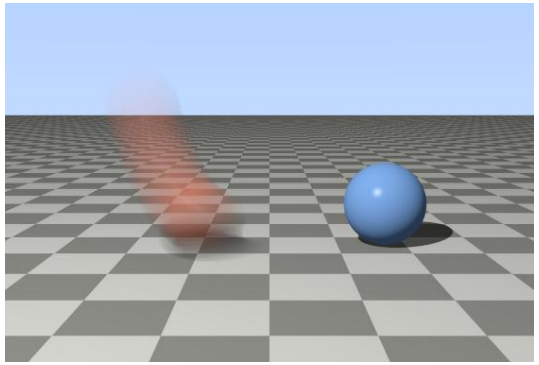
$$\tau = t_0 + \xi\,(t_1 - t_0), \qquad \mathbf{c}(\tau) = \mathbf{c}_0 + \frac{\tau - t_0}{t_1 - t_0}\,(\mathbf{c}_1 - \mathbf{c}_0), \qquad \text{raggio } (\mathbf{o},\ \mathbf{d},\ \tau)$$

5 · Che cosa costruire

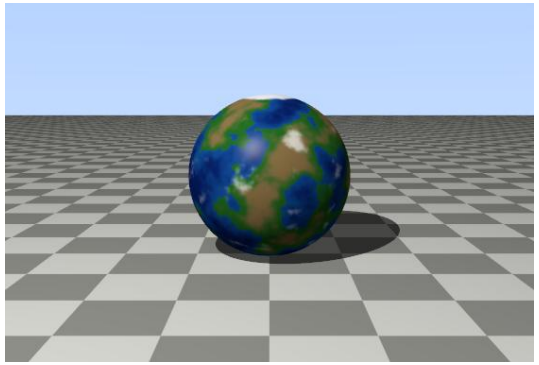
- Una sfera che rimbalza accanto a una ferma, che deve restare nitida; poi una sfera con texture che ruota sul proprio asse: la scia segue i paralleli
- Nel progetto della lezione 10 il BVH di TinyBVH è per mesh, nell'object space: un'istanza che si muove non chiede altro. Con il TLAS di `BVH_Scene` invece il box dell'istanza deve contenere tutto il moto



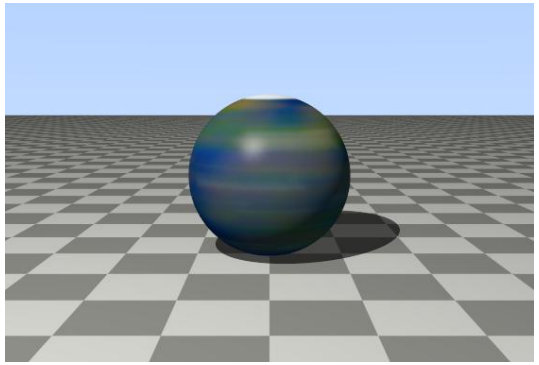
ferma



rimbalzo, 48 istanti



ferma



80° di rotazione durante la posa

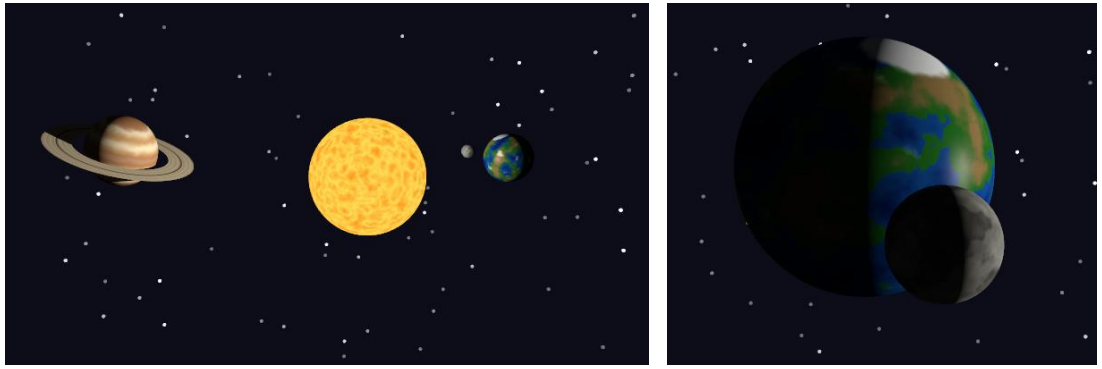
Che cosa consegnare

- il campo τ in `ray`, il centro (o la trasformazione) che dipende dal tempo e l'intervallo $[t_0, t_1]$ nella camera
- i quattro render, fermi e mossi, con almeno 32 campioni per pixel, e il tempo di calcolo di ciascuno

6 · Un sistema solare

●●○ medio

- Sole, terra, luna e un pianeta gigante con l'anello, con texture equirettangolari: servono le (*u*, *v*) della sfera (homework 8)
- Il sole non viene illuminato, **emette**: un materiale che restituisce il colore della texture senza Phong
- Trappola: la luce sta al centro del sole, quindi lo shadow ray di ogni pianeta colpisce la superficie del sole e tutto resta al buio. Il sole va escluso dagli shadow ray
- La rotazione della terra mostra il giorno e la notte, la luna mostra le fasi



un fotogramma e la terra da vicino, con la luna

Che cosa consegnare

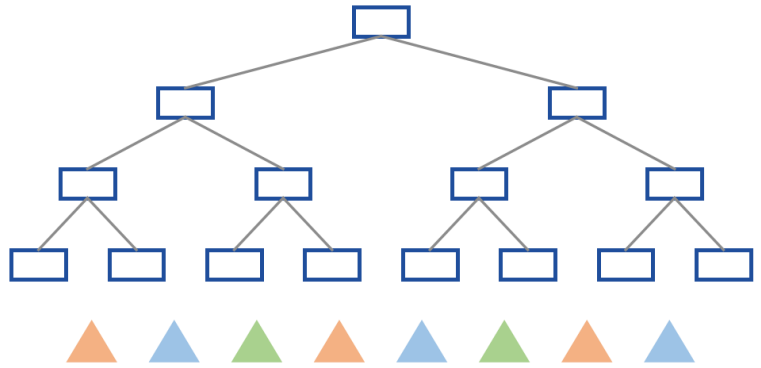
- Un video di almeno 20 secondi

7 · Un milione di conigli: due organizzazioni

●●○ medio

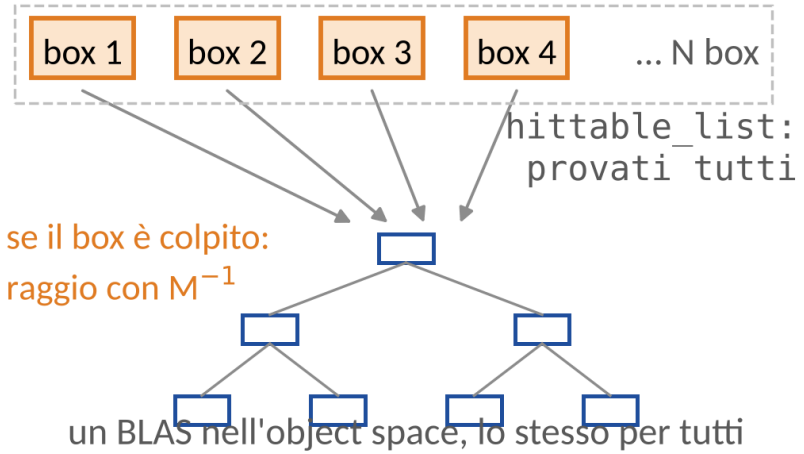
- La stessa scena, *N* conigli (`BunnyTiny`) con posizione, rotazione e scala casuali, resa in due modi a partire dal progetto della lezione 10
- **A · Un solo BLAS**: i triangoli di tutti i conigli, già trasformati nel world space, in un'unica `mesh`; TinyBVH costruisce un solo BVH su tutti. Ogni raggio visita un solo albero
- **B · Box e BLAS**: ogni coniglio è un'istanza con il suo bounding box nel world space (il box degli 8 vertici trasformati, lezione 9). Il raggio interseca prima il box; solo se lo colpisce passa nell'object space e visita il BLAS del coniglio
- In B i triangoli sono in memoria una volta sola, perché i conigli sono uguali e il BLAS si condivide; i box stanno in una `hittable_list` e ogni raggio li prova tutti

A · un solo BLAS



i triangoli di tutti i conigli, copiati nel world space

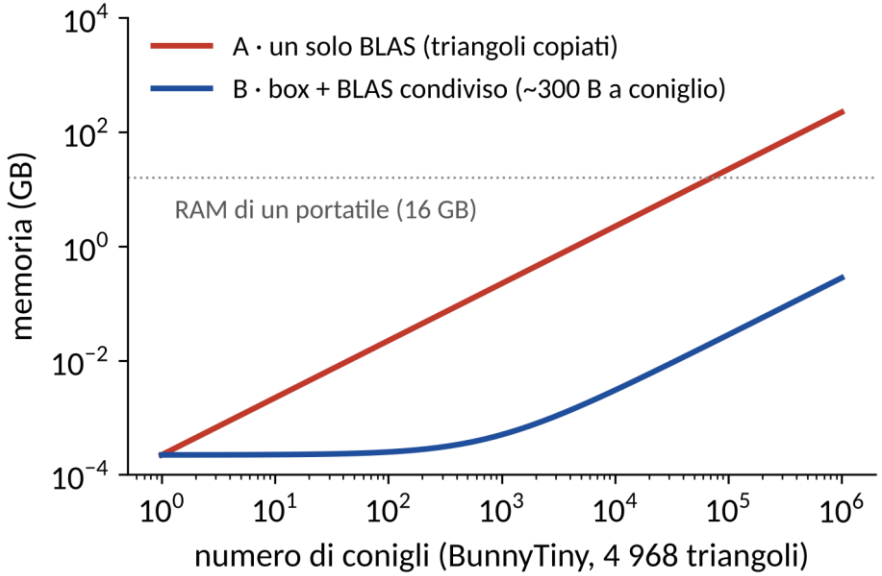
B · un box per coniglio, poi il suo BLAS



un BLAS nell'object space, lo stesso per tutti

7 · Quanta memoria occupano A e B

- *N* è il numero di conigli, *F* il numero di triangoli di un coniglio (4 968 per `BunnyTiny`)
- Un triangolo occupa **48 byte**: 3 vertici, ciascuno un `bvhvec4` di TinyBVH (4 `float` da 4 byte = 16 byte), come `tri_vertices` in `mesh.h`
- **A** copia i triangoli di ogni coniglio: la memoria cresce come $N \cdot F$
- **B** tiene i triangoli una volta sola; per ogni coniglio c'è solo l'istanza: le due matrici M e M^{-1} ($2 \cdot 64 = 128$ byte), il box (24 byte), puntatori e materiale, circa **300 byte**
- Sono stime dal basso: i nodi del BVH aggiungono memoria (in A su tutti i triangoli). Vanno confrontate con la memoria misurata



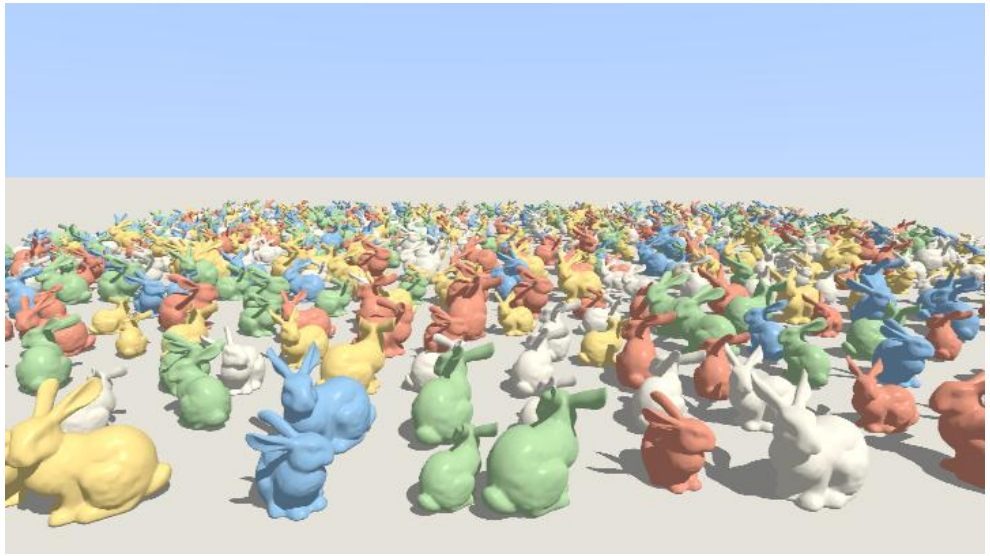
$$\text{A:}\quad \underbrace{N}_{\text{conigli}} \cdot \underbrace{F}_{\text{triangoli di un coniglio}} \cdot \underbrace{48}_{\text{3 vertici}\times\text{16 byte}} \ \text{byte}$$

$$\text{B:}\quad \underbrace{F \cdot 48}_{\text{un coniglio solo, condiviso}} + \underbrace{N \cdot \approx 300}_{\text{un'istanza per coniglio}} \ \text{byte}$$

$$F = 4968,\ N = 10^6: \qquad \text{A} \approx 222\ \text{GB} \qquad \text{B} \approx 0.24\ \text{MB} + 0.28\ \text{GB} \approx 0.28\ \text{GB}$$

7 · Che cosa misurare

- Per *N* = 10, 100, 1 000, 10 000, 100 000 e, se avete pazienza, 1 000 000, e per ciascuna organizzazione: tempo di costruzione, tempo di rendering, memoria del processo
- Che cosa aspettarsi: in A memoria e costruzione crescono con *N* (un milione di conigli non entra in un PC), il rendering cresce poco; in B la memoria resta quasi ferma ma il rendering cresce come *N*
- Dove si incrociano le due curve dei tempi? Che cosa manca a B per non provare tutti i box?



676 conigli: 3.4 milioni di triangoli nel solo BLAS di A

Che cosa consegnare

- A e B nello stesso programma, con un interruttore, e la tabella dei tempi e della memoria
- il grafico in assi logaritmici e una riga: perché un BVH sui box (il TLAS della lezione 9) unirebbe i vantaggi di A e di B?

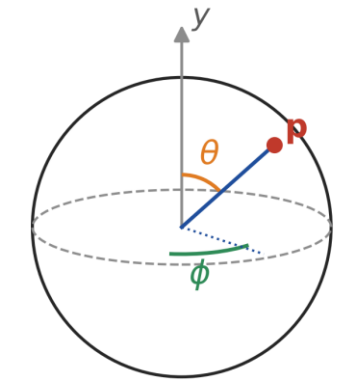
Homeworks

- Luci e ombre
- Trasformazioni e movimento
- **Texture e superfici**
- Il progetto finale
- Modalità di consegna

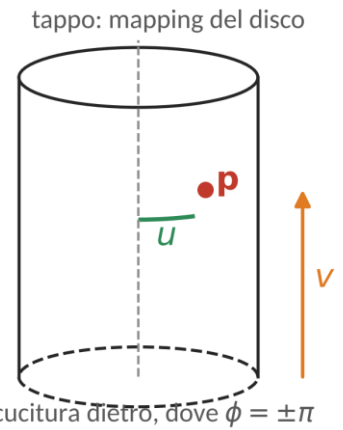
8 · Le coordinate di texture delle primitive

●○○ facile

- Le mesh hanno le (*u*, *v*) nel file; le primitive analitiche no: si calcolano nel metodo `hit` e si scrivono in `hit_record`
- **Sfera**: è l'esercizio 5 della lezione 10; qui si estende a tutte le primitive
- **Cilindro**: l'angolo attorno all'asse e l'altezza; sui tappi un mapping del disco
- **Quadrilatero**: (*α*, *β*) sono già coordinate di texture (homework 1 della I° serie); **box**: le (*α*, *β*) della faccia colpita
- Si calcolano nell'object space: con le istanze la texture segue l'oggetto



v = 1 al polo nord, 0 al polo sud



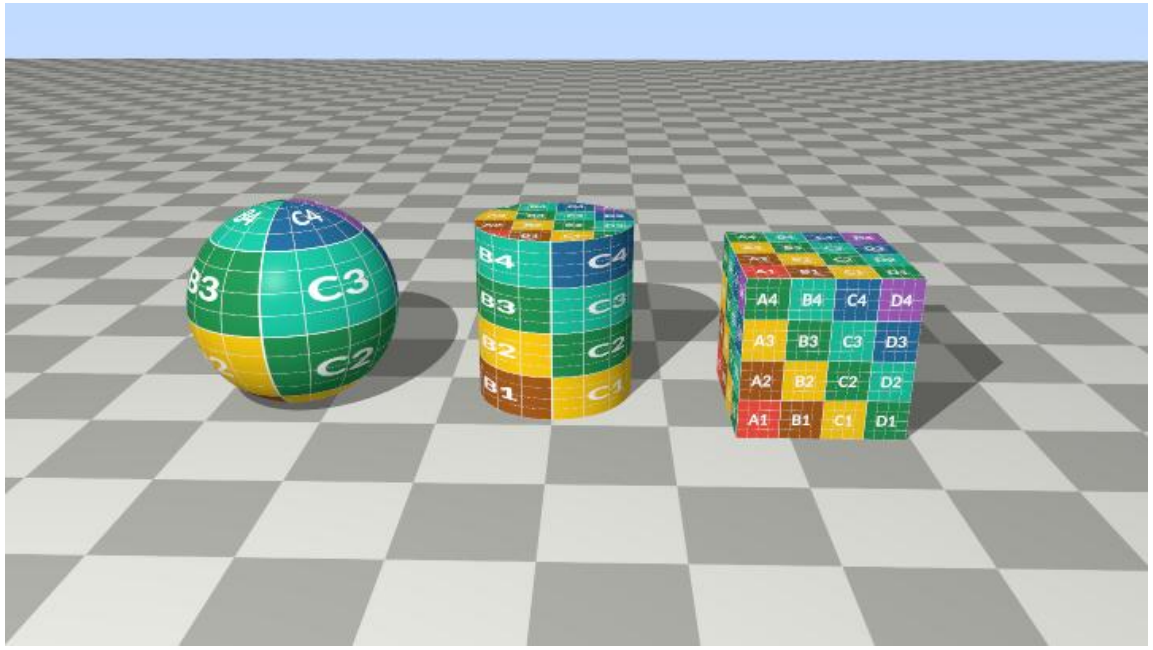
cucitura dietro, dove $\phi = \pm\pi$

$$\mathbf{q} = \frac{\mathbf{p} - \mathbf{c}}{r}, \quad \theta = \arccos(q_y), \quad \phi = \operatorname{atan2}(q_x, q_z), \quad u = \frac{1}{2} + \frac{\phi}{2\pi}, \quad v = 1 - \frac{\theta}{\pi}$$

$$u = \frac{1}{2} + \frac{\operatorname{atan2}(x - c_x,\ z - c_z)}{2\pi}, \qquad v = \frac{y - y_{\min}}{y_{\max} - y_{\min}}, \qquad \text{tappo: } (u, v) = \tfrac{1}{2} + \frac{(x - c_x,\ z - c_z)}{2r}$$

8 · Che cosa costruire

- Calcolare (*u*, *v*) in `sphere`, `cylinder` (con i tappi), `quad` e `box`. Il cilindro si prende dalla I° serie: `cylinder.h` della lezione 10 è una versione vecchia, senza tappi
- Prima delle foto, una texture a griglia con le lettere (A1…D4): si vede subito se *u* e *v* sono scambiati, rovesciati o se la cucitura è nel posto sbagliato
- Un fattore di ripetizione per *u* e *v* con il wrap (parte frazionaria): oggi `image_texture` blocca le coordinate al bordo
- Sui due tappi la stessa formula dà un'immagine specchiata su uno dei due: verificarlo con le lettere



la stessa griglia su sfera, cilindro con i tappi e box

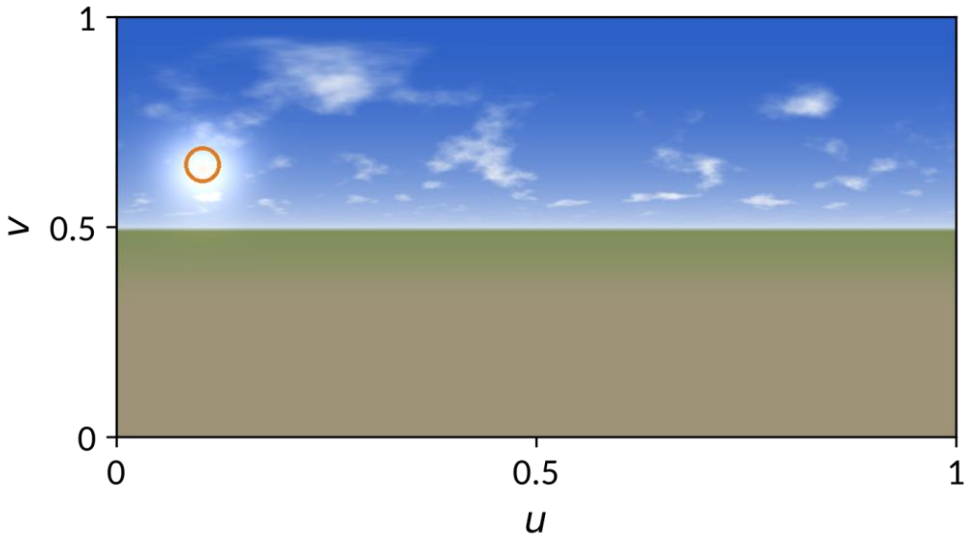
Che cosa consegnare

- le (*u*, *v*) delle quattro primitive e il fattore di ripetizione in `image_texture`
- il render con la griglia e uno con immagini a scelta, e una riga: dove sono le cuciture e i poli in ciascuna forma?

9 · Il cielo è una texture

●○○ facile

- Oggi lo sfondo è un gradiente; con un'immagine **equirettangolare** a 360° diventa un cielo vero
- Il raggio che non colpisce niente ha una direzione **d**: normalizzata, si trasforma in (*u*, *v*) come sulla sfera, senza centro né raggio
- Il cielo però si guarda **da dentro**: con la formula della sfera esce specchiato (le scritte si leggono al contrario), quindi *φ* cambia segno
- La luce deve venire da dove si vede il sole: il baricentro dei pixel più luminosi dà la direzione di una luce direzionale, e le ombre tornano coerenti con il cielo
- Le immagini HDR (.hdr) hanno valori oltre 1 e SDL_image non le legge: si usa un JPEG o un PNG (o stb_image)



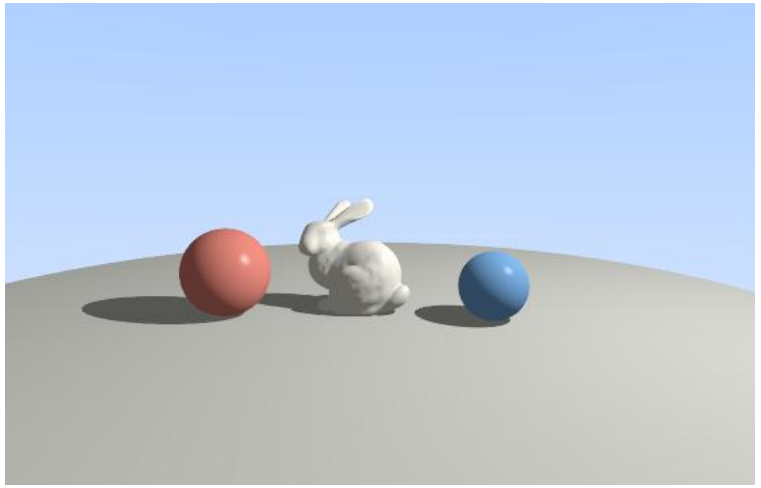
il cielo usato qui: nel cerchio arancio il sole

$$\hat{\mathbf{d}} = \frac{\mathbf{d}}{\|\mathbf{d}\|}, \quad \theta = \arccos(\hat{d}_y), \quad \phi = \operatorname{atan2}(-\hat{d}_x,\ \hat{d}_z), \quad u = \frac{1}{2} + \frac{\phi}{2\pi}, \quad v = 1 - \frac{\theta}{\pi}$$

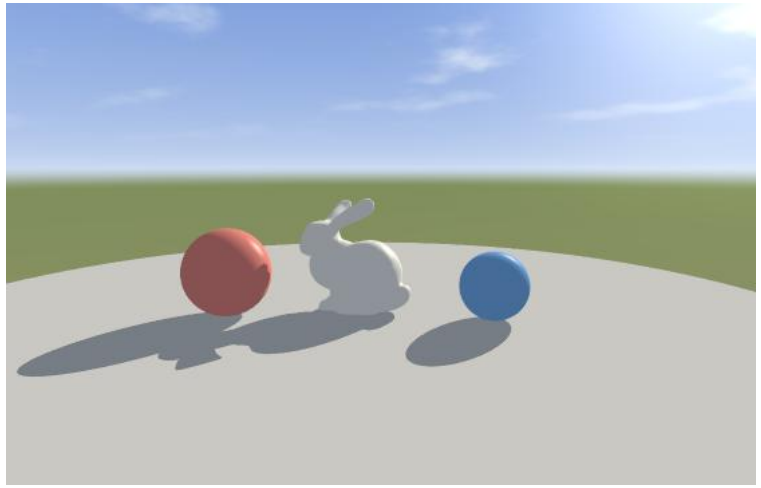
$$\text{inversa (dal pixel del sole alla direzione): } \hat{\mathbf{d}} = (-\sin\theta\sin\phi,\ \cos\theta,\ \sin\theta\cos\phi)$$

9 · Che cosa costruire

- Una `environment_texture` letta per direzione e usata in `ray_color` al posto del gradiente; la direzione del sole stimata dall'immagine
- Facoltativo: ruotare il cielo attorno a *y* con un angolo; spostando il sole cambia l'ora del giorno, e con lei le ombre



gradiente e luce puntiforme (lezione 10)



cielo e luce dal sole: le ombre vengono verso di noi

Che cosa consegnare

- `environment_texture` e la stima della direzione del sole, con l'errore in gradi se la direzione vera è nota
- due render della stessa scena: con il gradiente e la luce puntiforme, con il cielo e la luce direzionale

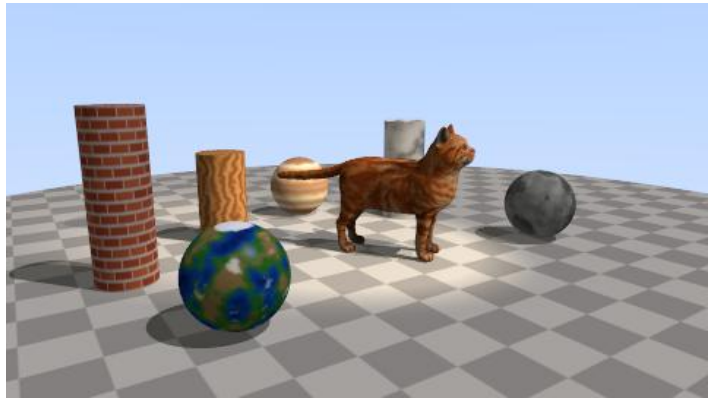
Homeworks

- Luci e ombre
- Trasformazioni e movimento
- Texture e superfici
- **Il progetto finale**
- Modalità di consegna

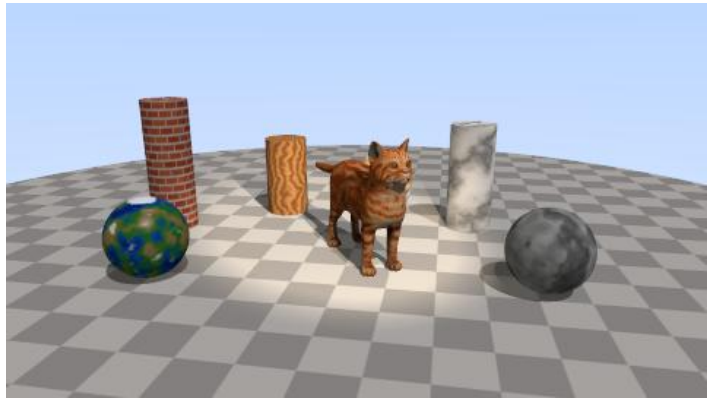
10 · Un cortometraggio

●●● difficile

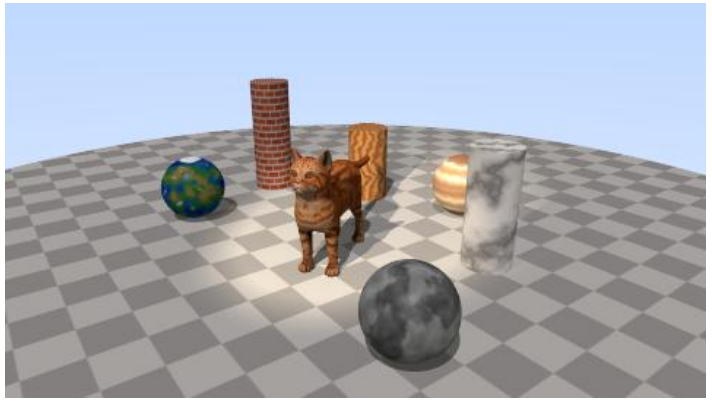
- Un video di almeno **30 secondi** con una scena che usa quello che avete costruito nelle due serie
- Nella scena: tre sfere e tre cilindri con texture, un modello OBJ con la sua texture, una luce direzionale e uno spot
- Almeno un oggetto in movimento (le trasformazioni di un'istanza che cambiano nel tempo) e la camera su un percorso (homework 10 della I° serie)
- Oltre all'1, al 3 e all'8, che servono già per la scena, almeno altri due homework di questa serie: proiettore, ombre morbide, toon, motion blur, cielo…
- Una storia, anche minima: un inizio, qualcosa che succede, una fine



0 s



15 s



30 s

10 · Il budget di tempo

- Con TinyBVH le mesh costano poco; quello che pesa sono i campioni per pixel e gli shadow ray, soprattutto con le ombre morbide
- Si prova il movimento a bassa risoluzione e con un campione, e solo alla fine si fa il rendering vero
- Il programma salva i fotogrammi numerati dalla matrice di colori di `parallel_render`, non rileggendo la finestra; il video si monta dopo
- Il BVH delle mesh si costruisce una volta; per far muovere un oggetto bastano le matrici della sua istanza

$$30\ \text{s} \times 25\ \text{fps} = 750\ \text{fotogrammi} \qquad \text{a } 2\ \text{s ciascuno: } 750 \times 2\ \text{s} = 25\ \text{minuti}$$

```
// nel ciclo dei fotogrammi
char name[64];
snprintf(name, sizeof name, "frame_%04d.bmp", k);
save_frame(matrix, name);   // non la finestra
```

```
ffmpeg -framerate 25 -i frame_%04d.bmp
       -pix_fmt yuv420p film.mp4
```

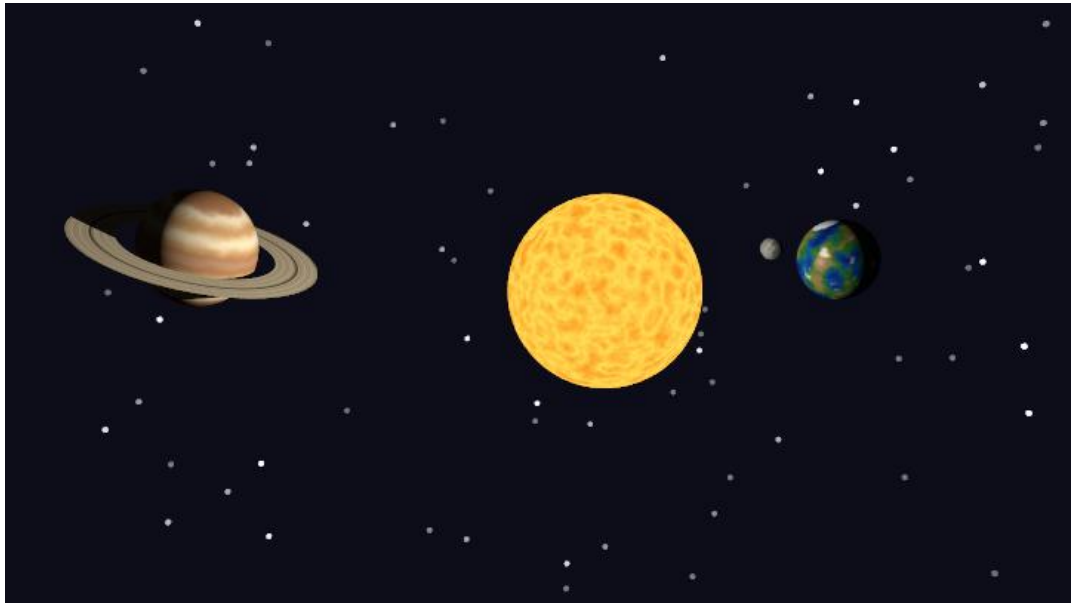
da fotogrammi numerati a video

Che cosa consegnare

- il video (MP4, almeno 960 × 540), il codice che genera i fotogrammi e un README con la scaletta e gli homework usati
- il tempo medio per fotogramma, il numero di campioni per pixel e la macchina usata

Fuori concorso: il fotogramma più bello

- Con luci, ombre, texture, mesh e istanze il renderer fa ormai immagini vere
- Chi vuole consegna **una sola immagine** in più, senza vincoli di scena o di risoluzione
- Può essere un fotogramma del cortometraggio o una scena a parte
- Le tre più votate in aula apriranno l'ultima lezione del corso



un sole, due pianeti, una luna e una sola luce

Una regola sola

- L’immagine o il video deve uscire dal vostro renderer

Consigli pratici

- Partite dal progetto della lezione 10 e tenetene una copia funzionante prima di toccare il codice
- Una luce o una texture nuova è una classe nuova: `ray_color` non deve diventare un elenco di `if`
- Luci: provate un solo shadow ray con la luce in una posizione in cui sapete già dove cade l'ombra
- Texture: la griglia con le lettere prima della foto; se l'immagine è specchiata, guardate il segno di u o la riga $1 - v$
- Animazioni: fotogrammi numerati e ffmpeg; il tempo si misura con `std::chrono::steady_clock`, come nel `main` della lezione 10
- Tutto quello che si muove si calcola da t: posizione, rotazione, camera. Così un fotogramma qualunque si rifà da solo

Se vi bloccate

- colorate il pixel con il valore sospetto: $\mathbf{n} \cdot \mathbf{l}$, il fattore dello spot, $(u, v, 0)$, la visibilità V. Un'immagine dice più di cento `printf`

Homeworks

- Luci e ombre
- Trasformazioni e movimento
- Texture e superfici
- Il progetto finale
- **Modalità di consegna**

Modalità di consegna

- Consegna solo attraverso la piattaforma Moodle
- Scadenza: 7 novembre
- Per ogni homework consegnare:
  - i sorgenti, cioè il progetto Visual Studio completo, senza le cartelle x64, Debug, Release e packages
  - l'eseguibile
  - i render richiesti nella scheda dell'homework e, dove richiesto, il video
  - un README.txt di una pagina: che cosa avete implementato, che cosa non funziona, dove vi siete bloccati
- Un archivio per homework, con nome Cognome_Nome_HW2_n.zip; i video grandi con un link
- Si discute tra di voi, ma si lavora e si consegna singolarmente

Come viene valutato

- che la parte nuova sia corretta, compresi i casi limite (luce dietro la superficie, punto fuori dal cono, cucitura della texture) — non che la scena sia complicata
- che il codice si integri nel renderer esistente invece di duplicarlo
- che il README dica la verità su quello che non funziona